

week09报告

所有相关文件可以在 [这里](#) 找到。

所有图片中的内容，包括代码、控制台输出，均可以在该文档中找到。

任务一：实现一个泛型容器类 `MyBox<T>`

MyBox.java

```
public class MyBox<T> {
    private T item;

    public void set(T item) {
        this.item = item;
    }

    public T get() {
        return item;
    }

    @Override
    public String toString() {
        return "MyBox{" +
            "item=" + item +
            '}';
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        MyBox<?> other = (MyBox<?>) o;
        if (item == null) {
            return other.item == null;
        }
        return item.equals(other.item);
    }
}
```

TestMyBox.java

```
public class TestMyBox {
    public static void main(String[] args) {
        MyBox<String> stringBox = new MyBox<>();
        stringBox.set("Hello, World!");
        System.out.println(stringBox.toString());

        MyBox<Integer> integerBox = new MyBox<>();
        integerBox.set(123);
        System.out.println(integerBox.toString());
    }
}
```

```

        MyBox<Double> doubleBox = new MyBox<>();
        doubleBox.set(3.14);
        System.out.println(doubleBox.toString());

        MyBox<String> anotherStringBox = new MyBox<>();
        anotherStringBox.set("Hello, World!");

        System.out.println("String Box equals anotherStringBox: " +
stringBox.equals(anotherStringBox));
        System.out.println("String Box equals integerBox: " + stringBox.equals(integerBox));
        System.out.println("String Box equals doubleBox: " + stringBox.equals(doubleBox));
    }
}

```

TestMyBox运行结果:

```

/home/fqwqf/.jdk/openjdk-24/bin/java -javaagent:/home/fqwqf/桌面/idea-IC-
243.25659.59/lib/idea_rt.jar=33417 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
Dsun.stderr.encoding=UTF-8 -classpath /home/fqwqf/桌
面/project/WHU_OOP_2025/week09/out/production/week09 TestMyBox
MyBox{item=Hello, World!}
MyBox{item=123}
MyBox{item=3.14}
String Box equals anotherStringBox: true
String Box equals integerBox: false
String Box equals doubleBox: false

进程已结束, 退出代码为 0

```

任务二：设计一个使用泛型接口和方法的集合处理器

Processor.java

```

public interface Processor<T> {
    void process(T item);
}

```

StringPrinter.java

```

public class StringPrinter implements Processor<String> {
    @Override
    public void process(String item) {
        System.out.println("Printing: " + item);
    }
}

```

NumberSummer.java

```

public class NumberSummer implements Processor<Integer> {
    private int sum = 0;

    @Override
    public void process(Integer item) {
        sum += item;
    }

    public int getSum() {
        return sum;
    }
}

```

ProcessorUtil.java

```

import java.util.List;

public class ProcessorUtil {
    public static <T> void apply(List<? extends T> list, Processor<T> processor) {
        for (T item : list) {
            processor.process(item);
        }
    }
}

```

TestProcessor.java

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class TestProcessor {
    public static void main(String[] args) {
        List<String> strings = Arrays.asList("Hello", "World", "fQwQf");
        StringPrinter stringPrinter = new StringPrinter();
        System.out.println("Processing Strings:");
        ProcessorUtil.apply(strings, stringPrinter);

        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        NumberSummer numberSummer = new NumberSummer();
        ProcessorUtil.apply(numbers, numberSummer);
        System.out.println("Sum of Numbers: " + numberSummer.getSum());

        // 拓展内容测试
        List<Object> dest = new ArrayList<>();
        List<Integer> src = Arrays.asList(1, 2, 3);
        collect(dest, src);
        System.out.println("Collected list: "+dest);
    }

    // 拓展内容
}

```

```
public static <T> void collect(List<? super T> dest, List<T> src) {
    dest.addAll(src);
}

}
```

TestProcessor运行结果:

```
/home/fqwqf/.jdk/openjdk-24/bin/java -javaagent:/home/fqwqf/桌面/idea-IC-
243.25659.59/lib/idea_rt.jar=36553 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
Dsun.stderr.encoding=UTF-8 -classpath /home/fqwqf/桌
面/project/WHU_00P_2025/week09/out/production/week09 TestProcessor
Processing Strings:
Printing: Hello
Printing: World
Printing: fQwQf
Sum of Numbers: 15
Collected list: [1, 2, 3]

进程已结束, 退出代码为 0
```

总结

任务一：泛型容器类 `MyBox<T>`

1. 核心实现

- 泛型类定义：通过 `MyBox<T>` 实现通用数据存储，支持 `set`、`get`、`toString` 方法，确保类型安全。
- `equals` 方法：使用 `MyBox<?>` 作为参数类型，允许跨类型比较。处理了 `item` 为 `null` 的情况，并通过 `getClass()` 确保类型严格一致。
- 测试验证：通过 `TestMyBox` 测试不同类型（`String`、`Integer`、`Double`）的存储与比较，验证了跨类型比较返回 `false` 的逻辑。

关键点：

- `<?>` 通配符用于接受任意类型的 `MyBox` 实例，但类型检查（`getClass()`）确保只有相同类型的实例才能通过 `equals` 比较。
- `item` 为空时的处理避免了空指针异常，保证逻辑健壮性。

任务二：集合处理器与泛型接口

1. 接口与实现

- 泛型接口 `Processor<T>`：定义统一方法 `process`，支持多态处理。
- 具体实现类：
 - `StringPrinter`：直接输出字符串内容。
 - `NumberSummer`：累加整数总和，通过 `getSum` 获取结果。
- 工具类 `ProcessorUtil`：通过泛型方法 `apply` 遍历列表，调用 `process` 方法，使用 `<? extends T>` 支持协变类型（如 `List<Integer>` 传递给 `Processor<Number>`）。

2. 拓展内容

- **collect** 方法：使用 `<? super T>` 实现逆变，允许将 `List<T>` 复制到其父类型的列表（如 `List<Object>`），增强灵活性。

关键点：

- `<? extends T>` 用于生产者场景（读取数据），`<? super T>` 用于消费者场景（写入数据），体现了PECS原则（Producer-Extends, Consumer-Super）。
- `NumberSummer` 通过内部状态（`sum` 变量）保存处理结果，需注意线程安全问题（本次实现未涉及多线程）。

测试结果与验证

- **TestMyBox** 输出：
不同类型实例的 `toString` 正确输出内容，跨类型比较返回 `false`，同类型内容相同返回 `true`。
- **TestProcessor** 输出：
 - `StringPrinter` 逐条输出字符串。
 - `NumberSummer` 正确累加整数总和。
 - `collect` 方法成功将 `List<Integer>` 复制到 `List<Object>`。